



Ernst & Young Advisory Pte. Ltd.
One Raffles Quay
North Tower, Level 18
Singapore 048583

Mailing Address:
Robinson Road
PO Box 384
Singapore 900734

Tel: +65 6535 7777
Fax: +65 6532 7662
ey.com

Qatar National Bank (Q.P.S.C.)

EY Blockchain – Smart Contracts Assessment

September 4, 2025





**Shape the future
with confidence**

To the management of QATAR NATIONAL BANK (Q.P.S.C.)

Ernst & Young Advisory Pte. Ltd. ('EY') have finalized our engagement to perform a assessment and deliver a final report ("Deliverable") only for QATAR NATIONAL BANK (Q.P.S.C.) ("Client") pursuant to an agreement solely between EY and Client. EY did not perform its services on behalf of or to serve the needs of any other person or entity. Accordingly, EY expressly disclaims any duties or obligations to any other person or entity, beyond the Client for whole this Deliverable is prepared, based on its use of the attached Deliverable. Any other person or entity must perform its own due diligence inquiries and procedures for all purposes, including, the appropriateness of the Client's consideration for any particular situation addressed by the Deliverable. This engagement has been performed in accordance with our agreement with the procedures being limited to those described in that agreement.

The procedures that we performed were advisory in nature and do not constitute an audit, review, examination or other form of attestation (as those terms are identified by the American Institute of Certified Public Accountants or by the Public Company Accounting Oversight Board) of Client's financial statements, including non-financial information, such as environmental, social and governance. Accordingly, EY did not express any form of assurance. The observations relating to assessment of the smart contracts that EY provided were: 1) based on the facts and circumstances presented to EY; 2) designed to assist the Client in reaching its own conclusions; and 3) do not constitute our concurrence with or support of the Client's decision, response or action.

EY did not conclude on the appropriate treatment based on specific facts or recommend which policy/treatment Client should select or adopt.



**Shape the future
with confidence**

The observations relating to the matters that EY provided to Client were designed to assist Client in reaching its own conclusions and do not constitute our concurrence with or support of Client's reporting. Client alone is responsible for any decisions to implement actions identified in our Deliverables or other actions from the provision of our services and for compliance with applicable regulatory requirements.

This information is not intended or written to be used, and it may not be used, for the purpose of avoiding penalties that may be imposed on a taxpayer.

As the agreement stipulates, EY has performed an assessment of the security, functionality, efficiency, and code quality of the smart contracts source code provided by Client. A final version, dated Aug 25, 2025, was then sent for retesting. This final report contains our conclusive findings from the assessment we performed and the findings that closed on the final version.

Given that our work is limited in scope and time, we stress that more detailed procedures may reveal issues that this engagement has not. The procedures summarized in this report do not constitute an audit or other form of assurance, or otherwise performed in accordance with any generally accepted auditing or other assurance standards, and thus we do not express any such commitment.

The comments, opinions, and suggestions stated in this report regarding the capabilities of the smart contract, whether expressed as being those of EY or not, are based on the limited information provided to EY by Client. While EY does not have reason to believe the information in this report is in any way misleading, inaccurate, or incomplete, the sole responsibility for the accuracy and completeness of the information resides with Client.



**Shape the future
with confidence**

Restrictions on the Use of Our Deliverable

Consistent with our engagement agreement, this report is intended solely for the information and use of the management of QATAR NATIONAL BANK (Q.P.S.C.). It is not intended to be, and should not be, used by anyone other than the parties specified here. We thank you for your collaboration enabling our delivery.

A handwritten signature in black ink, appearing to read 'Alvin Song', with a stylized flourish at the end.

Sincerely,

Ernst & Young Advisory Pte Ltd



Shape the future
with confidence

Table of Contents

Executive Summary	7
1. Introduction	7
2. Scope and Limitations	7
Assessment Methodology	8
Findings By Severity	10
MyToken.sol findings	11
1. Loss of control over the contract if the Owner accidentally executes the renounceOwnership function	11
2. Loss of control over the contract in case the ownership mistakenly transferred to an incorrect address	13
3. Non-owner is able to grant the fund manager role	15
4. Admin should not be able to renounce himself	17
5. Admin should not be able to revoke himself	19
6. Tokens sent to the contract may be permanently lost	21
7. Users who are not in the whitelist can transfer tokens to others	24
8. User may lose tokens due to an Address Poison Attack	26
9. Users who are not in the whitelist can transfer tokens to others	27
10. Admin should not be able to revoke permission from someone who don't have the permissions	28
11. Country restrictions are not documented	30
12. Checking for country restrictions is not efficient	32
13. Cannot add/remove countries during the lifetime of the smart contract	33
14. Not using the latest Solidity version	34



**Shape the future
with confidence**

15. The unnecessary variable <code>_lastRequestID</code> is wasting gas and not needed	35
16. The <code>recoverEther</code> function is unnecessary	36
17. Code is not documented well.....	37
<i>IdentifyRegistry.sol findings</i>	38
18. Admin should not be able to renounce himself.	38
19. Admin should not be able to revoke himself.	40
20. Wallets deletion fails with large number of wallets	42
21. Identity deletion fails with large number of wallets	44
22. Tokens sent to the contract may be permanently lost	46
23. Admin should not be able to revoke permission from someone who don't have the permissions.....	48
24. Duplicate admin removal functions may cause confusion.....	50
25. Not using the latest Solidity version	51
26. The <code>recoverEther</code> function is unnecessary	52
27. Code is not documented well.....	53



Shape the future
with confidence

Executive Summary

1. Introduction

This report serves to deliver an assessment of the security, functionality, efficiency, and overall quality of the smart contracts, to be used a fund which Client is managing as a sub-investment manager.

2. Scope and Limitations

This report does cover only the source code provided on August 25, 2025. Any modifications made to the source code or after that, are not covered by and/or taken into consideration in this report. Hence, modifications to the source code thereafter may affect the results of a future assessment and may therefore also change the nature of the recommendations provided in this report. QATAR NATIONAL BANK (Q.P.S.C.) provided final version for retesting on August 25, 2025. This version has partially assessed only for confirming fixes for issues found in the draft report of the first version.

Finally, our work is limited only to the smart contract source code, This report only concerns the source code SHA of 7afe and the fixing introduces on SHA a021.

As such, EY takes no responsibility over, and does not claim to have performed an assessment on, any version of the source code or its technical specification which may be different from the specific, and date and time-sensitive, versions stipulated in this report.

This report covers the results of a full assessment of the source code for security, functionality, efficiency, and overall quality, including the following scope of work:



Shape the future
with confidence

Assessment Methodology

The assessment consists of a testing methodology that involves both a manual and/or automated code assessment as well as static analysis tools. The manual and/or automated assessment and in-depth analysis of the underlying code are based on the source code's functionality, writing style, variable declarations, deep-loops, edge-cases handling, algorithms, and variable modifiers.

The assessment's objective is to identify issues in the code that present security, functionality and compatibility, efficiency, or quality related issues. Such issues are then presented according to their technical risk level, issue type, and other parameters relevant for evaluation. Furthermore, the identification of issue types can be understood as follows:

Security issues present vulnerabilities that may carry significant risk, such as unauthorized access, loss of funds, arithmetic over or under flows, or inadequate code usage.

Functionality issues present pitfalls that prevent execution according to the planned flow described in the technical specification of the source code and which can ultimately break usability by entering a functional deadlock state.

Efficiency issues present wasteful usage of the smart contract's computational resources, known as gas usage, which can lead to an increase of the total fee that is required to execute the smart contract in a blockchain network.



**Shape the future
with confidence**

The following procedures are key to the successful performance of a smart contract assessment, and have been implemented in the assessment of the smart contract assessed in this report:

Best practices for blockchain:

Applying knowledge regarding best practices with the purpose of addressing known issues from various angles including, but not limited to, security considerations, a survey of well-known attacks on smart contracts, and the use of references to community-based topics and discussions about both common use cases and uncommon edge cases.

Publicly available tools for analysis and evaluation:

Using public tools to spot security concerns in identified use cases and edge cases.

Manual and/or automatic code assessment of the source code:

Assessing the overall code quality with the aim of tracing vulnerabilities.

Business logic verification of the code in comparison with technical specification:

Verifying business logic based on the designed or expected behavior and functionality with the purpose of serving the business outcomes tied to the smart contract.

Efficient application of the code for optimal execution of the smart contract:

Evaluating the given source code's levels of efficiency and providing recommendations to resolve potential efficiency pitfalls in the execution of the smart contract.



Shape the future
with confidence

Findings By Severity

Severity	Description	Total	Fixed	Closed
Critical	Findings reveal vulnerabilities which may result in loss of control over the smart contract.	7	5	2
High	Findings reveal a serious vulnerability that could result in a loss of funds, compromise of the wallet integrity, or unauthorized access or compromise of administrative accounts or other system functions.	7	1	6
Medium	Finding reveals cases where the code fails to meet the specifications or does not follow leading practices, or where the gas cost can be significantly reduced.	11	9	2
Low	Findings that would not lead to a direct attack to the smart contract, but whose existence may confuse the users, the clients, other developers, or third parties who will use the smart contract.	2	2	0
Total		27	17	10

Note on the status of each finding:

For each finding, if the status is 'fixed', this means that the original finding no longer applies as it is either resolved or removed through code changes. If the status is 'closed', then this means the finding is still present but are closed based on client's response with their technology partner. Client may decide not to resolve them as they either do not find it relevant or they may have other ways to resolve the findings fully or partially that is outside the scope of this smart contract review engagement for which we are not able to verify.



Shape the future
with confidence

MyToken.sol findings

1. Loss of control over the contract if the Owner accidentally executes the renounceOwnership function

ID: C01

Severity

Critical

Description

This contract contains the renounceOwnership function, which may be unsafe to execute.

Once the contract owner accidentally runs this function, which requires no parameters, they immediately lose access to the contract.

As a result, the contract may end up without any control, and admin access to it will be lost.

Recommendation

Consider removing this functionality. If you wish to transfer control to another manager, you can always use the transferOwnership function.

Status

Fixed

Client Response with Technology Partner

The original contract design included three roles: Owner, Admin, and Fund Manager. Given the current limited functionality, we have decided to simplify the design to only Admin and Fund Manager, using a single AccessControl-based permission system and removing Ownable entirely. As a result, the renounceOwnership-related risk will



**Shape the future
with confidence**

no longer exist in the final version.

Even in the original design, this issue should not be considered Critical because:
The Owner account is secured by a Fireblocks MPC wallet
A multi-layer approval policy is in place for execution
The chance of accidentally executing renounceOwnership is practically zero

Final Remarks

Fixed.



Shape the future
with confidence

2. Loss of control over the contract in case the ownership mistakenly transferred to an incorrect address

ID: C02

Severity

Critical

Description

If the ownership of the contract is mistakenly transferred to the wrong address, control over the contract may be permanently lost. This poses a critical risk, as no entity will be able to perform administrative actions or recover ownership once the transfer is completed.

Recommendation

Implement a two-step transfer process in which the new owner must explicitly confirm acceptance of ownership before the transfer is finalized.

Status

Closed

Client Response with Technology Partner

We acknowledge the concern regarding potential loss of control if ownership is transferred to the wrong address. In our final design, we have removed Ownable and adopted `AccessControl` with `AccessControlDefaultAdminRules` to manage permissions and admin transfers.

This implementation inherently provides a safer transfer process:

- The Default Admin role can be reassigned securely



**Shape the future
with confidence**

- Transfers follow the built-in delay and confirmation rules of AccessControlDefaultAdminRules, reducing the chance of accidental or incorrect transfers
- Multiple approval layers in our Fireblocks MPC setup further mitigate the risk

Retesting Status

The above code fixes the issue but introduces a new one. If a delay is configured (1 day) and the Admin wallet is compromised, and needs to be replaced, it is not possible to transfer control to another wallet without waiting.

We consider this functionality as dangerous (when disregarding external security mechanisms not assessed by us, such as the MPC Wallet).

Consider to always set the delay to zero days and update the procedures document, or alternatively update the Solidity code and remove the requirement to wait for an Admin wallet replacement.

Client Response with Technology Partner After Retesting

We accept the first approach - setting the delay to zero days and updating the procedures document and disabled `changeDefaultAdminDelay` and `rollbackDefaultAdminDelay` to prevent enable the delay feature. We originally implemented the time delay to allow reversibility in case of accidental transfers to wrong addresses. However, since the Fireblocks' approval workflow will provide security controls and prevent erroneous transfers, there is no need for this additional safety buffer

Final Remarks

Close based on Client's response without second round of retesting.



Shape the future
with confidence

3. Non-owner is able to grant the fund manager role

ID: C03

Severity

Critical

Description

According to the specification, there are only two roles: Owner and Fund Manager. However, the code also contains an "admin" role. When ownership is transferred, the new owner can pause/unpause the smart contract and perform other operations, but the admin can still grant and revoke roles.

Recommendation

Simplify the usage of Owner and Admin, and use only one permission system.

Implement a two-step transfer process in which the new owner must explicitly confirm acceptance of ownership before the transfer is finalized.

Status

Fixed

Client Response with Technology Partner

We have simplified the roles to only Admin and Fund Manager, removing the separate Owner role. Now, Admin is the sole role authorized to grant or revoke roles, including Fund Manager, under a single AccessControl system. This resolves the concern about non-owner granting roles.

Even if not changed, this is not a real issue. The original design intentionally included an extra Admin role for daily operational management, separate from the Owner. Given the limited contract functions, reducing the Owner role and keeping only Admin and Fund Manager is a more appropriate approach.



**Shape the future
with confidence**

Retesting Status

Non admin is not an owner anymore

Final Remarks

Fixed.



Shape the future
with confidence

4. Admin should not be able to renounce himself

ID: C04

Severity

Critical

Description

Admins are able to call the `renounceRole` function and lose access to the smart contract. If there is no other admin, control of the smart contract may be lost.

Recommendation

Check if there are any other admins before allowing a user to remove themselves, or remove this function entirely.

Status

Fixed

Client Response with Technology Partner

We acknowledge the concern regarding potential loss of control if ownership is transferred to the wrong address. In our final design, we have removed `Ownable` and adopted `AccessControl` with `AccessControlDefaultAdminRules` to manage permissions and admin transfers.

This implementation inherently provides a safer transfer process:

- The Default Admin role can be reassigned securely
- Transfers follow the built-in delay and confirmation rules of `AccessControlDefaultAdminRules`, reducing the chance of accidental or incorrect transfers
- Multiple approval layers in our Fireblocks MPC setup further mitigate the risk



**Shape the future
with confidence**

Final Remarks

Fixed



Shape the future
with confidence

5. Admin should not be able to revoke himself

ID: C05

Severity

Critical

Description

Admins are able to call the `renounceRole` function and lose access to the smart contract. If there is no other admin, control of the smart contract may be lost.

Recommendation

Check if there are any other admins before allowing a user to remove themselves, or remove this function entirely.

Status

Closed

Client Response with Technology Partner

We have implemented checks within the `revokeRole` function to prevent an admin from revoking their own role if they are the last remaining admin. This ensures the contract never loses all admin control.

Moreover, in actual operations, the risk of such accidental revocation is extremely low due to the multi-party approval process enforced by our Fireblocks MPC wallet.

Retesting Status

The code differs from your explanation: it does not allow an admin to remove themselves even if there are other admins.



**Shape the future
with confidence**

In your explanation, it was stated that self-removal is blocked only if the admin is the last one.

Regarding the protection by the MCP Wallet, in a smart contract assessment, we disregard external security in order to ensure that the contract itself maintains the highest protection level without relying on security mechanisms outside the defined scope.

Client Response with Technology Partner After Retesting

To clarify: the code correctly implements a policy where no role can revoke their own permissions under any circumstances - not just when they are the last admin. My earlier statement is to address original issue.

Final Remarks

Closed based on client feedback.



Shape the future
with confidence

6. Tokens sent to the contract may be permanently lost

ID: H01

Severity

High

Description

The smart contract does not have a function to recover tokens sent to it. If users mistakenly send ERC20 tokens or any other tokens to the smart contract, they will be lost.

Recommendation

Add a function that allows an admin to recover lost tokens.

Status

Closed

Client Response with Technology Partner

We agree that adding a token recovery function is a good-to-have feature to improve user experience. Although it's not a core responsibility of the contract, we are willing to implement this functionality as a precautionary measure to help recover tokens mistakenly sent to the contract.

Retesting Status

The fix allows recovery only of ERC20 tokens that implement the `safeTransfer` function.

Many tokens are not ERC20, and many ERC20 tokens do not implement `safeTransfer`. Consider to use more generic solution that works for more types of tokens (ERC721, ERC1155) by granting the admin permission to execute transactions on behalf of the contract.



**Shape the future
with confidence**

Example code:

```
function recoverTokens(  
    address target,  
    uint256 value,  
    bytes calldata data  
) external onlyRole(DEFAULT_ADMIN_ROLE) {  
    target.call{value: value}(data);  
}
```

This is only a demonstration for communication purposes. Choose the code that best fits your case.

Client Response with Technology Partner After Retesting

- SafeERC20 is a library wrapper, not a token method - It wraps standard ERC20 transfer() calls with compatibility handling. Tokens don't need to implement safeTransfer; the library calls their standard transfer() method internally.
- SafeERC20 actually improves compatibility - It handles both standard ERC20 tokens and non-standard ones (like USDT) that don't return boolean values, making it more compatible than using plain transfer().
- Regarding the generic solution - While the suggested approach is more flexible, it introduces significant security risks by allowing arbitrary calls. ERC20 recovery is sufficient for our use case as these are the most commonly mis-sent tokens. Recovery of other token types (ERC721/ERC1155) is outside our scope of responsibility. The suggested generic call function would introduce unnecessary security risks for functionality we don't intend to support.



**Shape the future
with confidence**

Final Remarks

The fix allows recovery only of ERC20 tokens, many tokens are not ERC20. Consider to use more generic solution that works for more types of tokens (ERC721, ERC1155) by granting the admin permission to execute transactions on behalf of the contract.



Shape the future
with confidence

7. Users who are not in the whitelist can transfer tokens to others

ID: H02

Severity

High

Description

When someone is in the whitelist, they can give access to another address through the approve mechanism. In this way, a non-whitelisted user can use the tokens of a whitelisted user and transfer them.

This means that unknown users can interact with the contract and transfer tokens, which creates a regulatory risk.

Recommendation

Consider adding a check in delegated transfers to ensure that the spender is also in the whitelist.

Status

Closed

Client Response with Technology Partner

While non-whitelisted addresses can be approved as spenders, our contract ensures that tokens can only be transferred to whitelisted addresses. This means regardless of who initiates the transfer, the tokens will always end up with verified, compliant addresses. Ensuring the final destination is whitelisted addresses the regulatory requirements, as this guarantees all token holders remain compliant entities. In addition, the transfer of tokens is a controlled action where the pre-approval of the



**Shape the future
with confidence**

Fund Administrator is required. Our control strategy combines both smart contract and external security to deliver the outcome we need and meets regulator's requirements.

Final Remarks

Closed based on client feedback.



Shape the future
with confidence

8. User may lose tokens due to an Address Poison Attack

ID: H03

Severity

High

Description

In this attack, the attacker sends 0 tokens to someone else from an address that looks similar to an address the victim recognizes, in order to confuse them. Later, when the victim wants to send tokens again to an address from their history, they might accidentally send them to the attacker's address instead of the correct one.

Recommendation

Consider disallow users to transfer zero tokens on behalf of others.

Status

Fixed

Client Response with Technology Partner

We added validation in `_update` function.

- `require(value > 0, "ERC20: transfer amount must be greater than zero")`;

In addition, the transfer of tokens is a controlled action where the pre-approval of the Fund Administrator is required. Our control strategy combines both smart contract and external security to deliver the outcome we need and meets regulator's requirements.

Final Remarks

Retested and fixed.



Shape the future
with confidence

9. Users who are not in the whitelist can transfer tokens to others

ID: H04

Severity

High

Description

In case of adding to the whitelist a smart contract, users who are not whitelisted can use the smart contract for transferring tokens even that they are not whitelisted. It might be against the policy requirements and might create regulatory issues.

Recommendation

To be sure that only whitelisted users are transferring tokens, consider check the tx.origin in addition to msg.sender.

Status

Closed

Client Response with Technology Partner

Prohibiting smart contracts would prevent clients from using multi-sig wallets, which are essential for institutional security. Ensuring the final destination is whitelisted addresses the regulatory requirements, as this guarantees all token holders remain compliant entities. In addition, the transfer of tokens is a controlled action where the pre-approval of the Fund Administrator is required. Our control strategy combines both smart contract and external security to deliver the outcome we need and meets regulator's requirements.

Final Remarks

Closed based on client feedback.



Shape the future
with confidence

10. Admin should not be able to revoke permission from someone who don't have the permissions

ID: M01

Severity

Medium

Description

Best practice dictates that when calling a function intended to modify the state, but the state does not actually change, the transaction should revert. In the case of the `revokeRole` function, if an incorrect address is entered by mistake, the current admin may believe that the intended manager was removed, while in reality the wrong address was provided and the targeted manager was not removed.

Recommendation

Update the function to revert if the provided address does not belong to a valid manager.

Status

Closed

Client Response with Technology Partner

This issue doesn't cause real problems, but we agree it's better for user experience. We will update the function to revert if trying to revoke a role from an address that doesn't have it.

Retesting Status

Fixed only for admin roles, but not for fund manager role. According to this findings transaction should revert when trying to revoke fund manager role from who don't have that permission.



**Shape the future
with confidence**

Client Response with Technology Partner After Retesting

Implemented the validation: require(

 hasRole(role, account),

 "Account does not have the role to be revoked"

);

Final Remarks

Fixed the original findings based on code change and client feedback.



Shape the future
with confidence

11. Country restrictions are not documented

ID: M02

Severity

Medium

Description

The specifications document does not describe any restrictions related to countries, but the smart contract has two modes: Allow list, and block list per country

Recommendation

Update the Specification and consider removing country information from the contracts.

Status

Fixed

Client Response with Technology Partner

The country restriction feature has been deprecated and removed from the smart contract. This decision was made because investor screening for restricted jurisdictions is conducted by our fund admin as part of the standard subscription workflow, thereby eliminating the need for duplicate compliance controls within our system.

Retesting Status

Currently, there is no restriction by country. However, the contract does require and store the country of each identity and also allows updating it.

This may confuse users and expose you to the risk of someone from a non-approved country explicitly using the contract.



**Shape the future
with confidence**

In addition, the Specification documents still define two Country modes: Blocklist and Allowlist.

Client Response with Technology Partner After Retesting

All country restriction functionality has been removed from the MyToken contract, leaving only the country field in the Identity Registry contract in case other token contracts need it in the future.

Final Remarks

Fixed the original findings based on code change and client feedback.



Shape the future
with confidence

12. Checking for country restrictions is not efficient

ID: M03

Severity

Medium

Description

Since every transfer is iterating over all the countries in a for...loop it's not efficient, take more resources/gas than necessary.

Recommendation

Consider using better storage - such as mapping.

Status

Fixed

Client Response with Technology Partner

All country restriction functionality has been removed from the MyToken contract, leaving only the country field in the Identity Registry contract in case other token contracts need it in the future.

Final Remarks

Fixed



Shape the future
with confidence

13. Cannot add/remove countries during the lifetime of the smart contract

ID: M04

Severity

Medium

Description

Solidity smart contracts are immutable. Although certain parameters, such as adding or removing identities or freezing accounts, can be modified, the list of countries remains fixed and cannot be updated.

Recommendation

Implement functions that allow adding and removing countries from the blacklist or allowed list.

Status

Fixed

Client Response with Technology Partner

All country restriction functionality has been removed from the MyToken contract, leaving only the country field in the Identity Registry contract in case other token contracts need it in the future.

Final Remarks

Fixed



Shape the future
with confidence

14. Not using the latest Solidity version

ID: M05

Severity

Medium

Description

In every new Solidity release, many bugs and security issues are fixed. Since the contract is not using the latest version, there may be vulnerabilities and bugs that could impact its behavior. No specific vulnerability was found, but best practice is to always use the latest stable version.

Recommendation

Upgrade the contract to the latest Solidity version to ensure all security patches and improvements are applied.

Status

Fixed

Client Response with Technology Partner

Upgraded to v0.8.30 and performed regression testing.

Final Remarks

Retested and fixed (Tested the client using 0.8.30).



Shape the future
with confidence

15. The unnecessary variable `_lastRequestID` is wasting gas and not needed

ID: M06

Severity

Medium

Description

The `mintRequest` and `burnRequest` functions assign and use `_lastRequestID` for a unique encoded Id, but it seems unnecessary since `requestCounter` is already unique. We didn't found reason for its existence, and no documentation explains it. It's making request take more gas, and not following the best practices.

Recommendation

Remove this storage variable to reduce costs and simplify the code.

Status

Fixed

Client Response with Technology Partner

Deleted this redundant field.

Final Remarks

Fixed

16. The recoverEther function is unnecessary

ID: M07

Severity

Medium

Description

Since the contract cannot receive Ether and has no payable functions, there is no possibility of stuck ETH. This does not create a security issue, but it may confuse developers, appear unprofessional, and goes against best practices.

Recommendation

Consider removing this function.

Status

Fixed

Client Response with Technology Partner

Deleted this redundant function.

Final Remarks

Retested and fixed



Shape the future
with confidence

17. Code is not documented well

ID: L01

Severity

Low

Description

The leading practice is that every function will be documented using the NatSpec format. This smart contract is almost undocumented, and this can cause a feeling of lack of reliability among users

Recommendation

Consider adding NatSpec format documentation to the code.

Status

Fixed

Client Response with Technology Partner

Cleaned up the code style and added comments.

Final Remarks

Fixed



Shape the future
with confidence

IdentifyRegistry.sol findings

18. Admin should not be able to renounce himself.

ID: C01

Severity

Critical

Description

Admins are able to call the `renounceRole` function and lose access to the smart contract. If there is no other admin, control of the smart contract may be lost.

Recommendation

Check if there are any other admins before allowing a user to remove themselves, or remove this function entirely.

Status

Fixed

Client Response with Technology Partner

We acknowledge the concern regarding potential loss of control if ownership is transferred to the wrong address.

In our final design, we have removed `Ownable` and adopted `AccessControl` with `AccessControlDefaultAdminRules` to manage permissions and admin transfers.

This implementation inherently provides a safer transfer process:

- The Default Admin role can be reassigned securely



**Shape the future
with confidence**

- Transfers follow the built-in delay and confirmation rules of AccessControlDefaultAdminRules, reducing the chance of accidental or incorrect transfers
- Multiple approval layers in our Fireblocks MPC setup further mitigate the risk

Retesting Status

renounceOwnership function removed.

Final Remarks

Fixed



Shape the future
with confidence

19. Admin should not be able to revoke himself.

ID: C02

Severity

Critical

Description

Admins are able to call the `revokeRole` function and lose access to the smart contract. If there is no other admin, control of the smart contract may be lost.

Recommendation

Check if there are any other admins before allowing a user to remove themselves, or remove this function entirely.

Status

Fixed

Client Response with Technology Partner

We acknowledge the concern regarding potential loss of control if ownership is transferred to the wrong address.

In our final design, we have removed `Ownable` and adopted `AccessControl` with `AccessControlDefaultAdminRules` to manage permissions and admin transfers.

This implementation inherently provides a safer transfer process:

- The Default Admin role can be reassigned securely
- Transfers follow the built-in delay and confirmation rules of `AccessControlDefaultAdminRules`, reducing the chance of accidental or incorrect transfers
- Multiple approval layers in our Fireblocks MPC setup further mitigate the risk



**Shape the future
with confidence**

Retesting Status

Admin is not able now to revoke his own permissions.

Final Remarks

Fixed



Shape the future
with confidence

20. Wallets deletion fails with large number of wallets

ID: H01

Severity

High

Description

If there are too many wallets, it will no longer be possible to delete wallets. The wallet deletion function iterates through all of the identity's wallets. If the number of wallets is too large, the loop may not complete, resulting in an out-of-gas error due to the block gas limit.

Recommendation

Consider switching to a data structure that does not require nested loops.

Status

Closed

Client Response with Technology Partner

In our contract design, each user is limited to a maximum of 5 wallet addresses, enforced at the time of contract deployment. Therefore, the scenario of having an excessively large number of wallets causing out-of-gas errors will not occur.

Retesting Status

We don't see the 5 wallets enforcement in the code. Consider add it as a require statement in the constructor function.



**Shape the future
with confidence**

Client Response with Technology Partner After Retesting

It will be set to 5 in the constructor function when deploying the contract. In addition, the Fund only allows each investors to list 5 wallets for whitelisting.

Final Remarks

Closed based on client feedback.



Shape the future
with confidence

21. Identity deletion fails with large number of wallets

ID: H02

Severity

High

Description

If there are too many wallets, it will no longer be possible to delete the identity. The identity deletion function iterates through all of the identity's wallets. If the number of wallets is too large, the loop may not complete, resulting in an out-of-gas error due to the block gas limit.

Recommendation

Consider implementing a hard limit in the code for the number of wallets allowed per identity.

Status

Closed

Client Response with Technology Partner

In our contract design, each user is limited to a maximum of 5 wallet addresses, enforced at the time of contract deployment. Therefore, the scenario of having an excessively large number of wallets causing out-of-gas errors will not occur.

Retesting Status

We don't see the 5 wallets enforcement in the code. Consider add it as a require statement in the constructor function.



**Shape the future
with confidence**

Client Response with Technology Partner After Retesting

It will be set to 5 in the constructor function when deploying the contract. In addition, the Fund only allows each investors to list 5 wallets for whitelisting

Final Remarks

Closed based on client feedback.



Shape the future
with confidence

22. Tokens sent to the contract may be permanently lost

ID: H03

Severity

High

Description

The smart contract does not have a function to recover tokens sent to it. If users mistakenly send ERC20 tokens or any other tokens to the smart contract, they will be lost.

Recommendation

Add a function that allows an admin to recover lost tokens.

Status

Closed

Client Response with Technology Partner

We agree that adding a token recovery function is a good-to-have feature to improve user experience. Although it's not a core responsibility of the contract, we are willing to implement this functionality as a precautionary measure to help recover tokens mistakenly sent to the contract.

Retesting Status

The fix allows recovery only of ERC20 tokens that implement the `safeTransfer` function.

Many tokens are not ERC20, and many ERC20 tokens do not implement `safeTransfer`. Consider to use more generic solution that works for more types of tokens (ERC721, ERC1155) by granting the admin permission to execute transactions on behalf of the contract.



**Shape the future
with confidence**

Example code:

```
function recoverTokens(  
    address target,  
    uint256 value,  
    bytes calldata data  
) external onlyRole(DEFAULT_ADMIN_ROLE) {  
    target.call{value: value}(data);  
}
```

This is only a demonstration for communication purposes. Choose the code that best fits your case.

Client Response with Technology Partner After Retesting

- SafeERC20 is a library wrapper, not a token method - It wraps standard ERC20 transfer() calls with compatibility handling. Tokens don't need to implement safeTransfer; the library calls their standard transfer() method internally.
- SafeERC20 actually improves compatibility - It handles both standard ERC20 tokens and non-standard ones (like USDT) that don't return boolean values, making it more compatible than using plain transfer().
- Regarding the generic solution - While the suggested approach is more flexible, it introduces significant security risks by allowing arbitrary calls. ERC20 recovery is sufficient for our use case as these are the most commonly mis-sent tokens. Recovery of other token types (ERC721/ERC1155) is outside our scope of responsibility. The suggested generic call function would introduce unnecessary security risks for functionality we don't intend to support.

Final Remarks

The fix allows recovery only of ERC20 tokens, many tokens are not ERC20. Consider to use more generic solution that works for more types of tokens (ERC721, ERC1155) by granting the admin permission to execute transactions on behalf of the contract.

Closed based on client feedback.



Shape the future
with confidence

23. Admin should not be able to revoke permission from someone who don't have the permissions

ID: M01

Severity

Medium

Description

Best practice dictates that when calling a function intended to modify the state, but the state does not actually change, the transaction should revert. In the case of the `revokeRole` function, if an incorrect address is entered by mistake, the current admin may believe that the intended manager was removed, while in reality the wrong address was provided and the targeted manager was not removed.

Recommendation

Update the function to revert if the provided address does not belong to a valid manager.

Status

Closed

Client Response with Technology Partner

This issue doesn't cause real problems, but we agree it's better for user experience. We will update the function to revert if trying to revoke a role from an address that doesn't have it.



**Shape the future
with confidence**

Retesting Status

Fixed only for admin roles, but not for fund manager role. According to this findings transaction should revert when trying to revoke fund manager role from who don't have that permission.

Client Response with Technology Partner After Retesting

implemented the validation: require(

 hasRole(role, account),

 "Account does not have the role to be revoked"

);

Final Remarks

Close based on Client's response without second round of retesting.



Shape the future
with confidence

24. Duplicate admin removal functions may cause confusion

ID: M02

Severity

Medium

Description

The removeAdmin and revokeRole functions perform similar actions, which can lead to user confusion and it's against the best-practices. Having multiple functions for the same purpose increases the risk of incorrect usage and may create unnecessary complexity. In the event of future upgrades, maintaining two separate implementations for the same functionality can result in inconsistencies and potential incompatibility issues.

Recommendation

Merge both functions into a single implementation to ensure clarity, reduce complexity, and prevent compatibility issues in future upgrades.

Status

Fixed

Client Response with Technology Partner

To keep the permission system simple, we have removed all additional permission management functions and retained only the functions provided by AccessControl.

Retesting Status

The removeAdmin function is removed.

Final Remarks

Fixed



Shape the future
with confidence

25. Not using the latest Solidity version

ID: M03

Severity

Medium

Description

In every new Solidity release, many bugs and security issues are fixed. Since the contract is not using the latest version, there may be vulnerabilities and bugs that could impact its behavior. No specific vulnerability was found, but best practice is to always use the latest stable version.

Recommendation

Upgrade the contract to the latest Solidity version to ensure all security patches and improvements are applied.

Status

Fixed

Client Response with Technology Partner

Upgraded to v0.8.30 and performed regression testing.

Final Remarks

Retested and fixed (Tested the the client using 0.8.30).



Shape the future
with confidence

26. The recoverEther function is unnecessary

ID: M04

Severity

Medium

Description

Since the contract cannot receive Ether and has no payable functions, there is no possibility of stuck ETH. This does not create a security issue, but it may confuse developers, appear unprofessional, and goes against best practices.

Recommendation

Consider removing this function

Status

Fixed

Client Response with Technology Partner

Deleted the function.

Final Remarks

Fixed after retesting.



Shape the future
with confidence

27. Code is not documented well

ID: L01

Severity

Low

Description

The leading practice is that every function will be documented using the NatSpec format. This smart contract is almost undocumented, and this can cause a feeling of lack of reliability among users

Recommendation

Consider adding NatSpec format documentation to the code

Status

Fixed

Client Response with Technology Partner

Cleaned up the code style and added comments

Final Remarks

Fixed



**Shape the future
with confidence**

Conclusion

In our assessment, we did not identify any direct method or attack that would allow someone to take control of the contract, steal funds, or alter permissions.

Given the high level of security required and the associated risks, there are multiple safety and protection mechanisms that could be added to the code.

As stated in the findings, most of the missing protection mechanisms we identified have been addressed.

There are also protective measures declared by Client outside of the smart contract code, through the wallet they use. These mechanisms were not examined and may address some of the issues presented or trigger other security issues.

For maximum security, we recommend fixing or addressing all of the findings.



**Shape the future
with confidence**

**EY | Assurance | Tax | Transactions |
Advisory**

About EY

EY is a global leader in assurance, tax, transaction and advisory services. The insights and quality services we deliver help build trust and confidence in the capital markets and in economies the world over. We develop outstanding leaders who team to deliver on our promises to all of our stakeholders. In so doing, we play a critical role in building a better working world for our people, for our clients and for our communities.

EY refers to the global organization, and may refer to one or more, of the member firms of Ernst & Young Global Limited, each of which is a separate legal entity. Ernst & Young Global Limited, a UK company limited by guarantee, does not provide services to clients. For more information about our organization, please visit ey.com.

© 2021 EYGM Limited.
All Rights Reserved.

This material has been prepared for general informational purposes only and is not intended to be relied upon as accounting, tax, or other professional advice. Please refer to your advisors for specific advice.